

Bike vs Bicycle Detection — CNN Image Classifier

A Convolutional Neural Network (CNN) built with TensorFlow/Keras that classifies images as either a motorbike or a bicycle. The project covers the full pipeline: loading images, augmenting a small dataset, training a CNN, evaluating it, and running predictions on new images.

Overview

Item	Value
Task	Binary image classification (bike vs bicycle)
Framework	TensorFlow / Keras
Input size	150 x 150 x 3
Raw dataset size	36 images
Augmented dataset size	756 images
Test accuracy	95.39%

Project Structure

```
.
├── Bike_or_Bicycle_Detection_CNN.ipynb    # Main notebook (data prep -> training ->
prediction)
├── README.docx
├── data/
│   ├── Bike/                             # Motorbike images
│   └── Bicycle/                           # Bicycle images
```

Note: The dataset itself is not included in this repo. Update the `base_folder` path in the notebook to point to your local Bike / Bicycle image folders.

Requirements

```
pip install tensorflow tf-keras opencv-python numpy scikit-learn albumentations
matplotlib
```

Pipeline

1. Data Loading

Images are read from two class subfolders (Bike, Bicycle) with OpenCV, resized to 150x150, and stacked into NumPy arrays.

2. Data Augmentation

Because the raw dataset is tiny (36 images), heavy augmentation is applied using Albumentations — each image is expanded into 20 augmented variants:

- Horizontal flip ($p=0.5$)
- Slight vertical flip ($p=0.1$ — bikes/bicycles are rarely upside down)
- Rotation ($\pm 15^\circ$, $p=0.5$)
- Random brightness/contrast ($p=0.5$)

- Gaussian noise (p=0.5)
- Median blur (p=0.3)

This grows the dataset from **36 → 756** images.

3. Preprocessing

- Labels are encoded with LabelEncoder (bicycle → 0, bike → 1)
- Pixel values are normalized to [0, 1]
- Data is split 80/20 into train/test sets (train_test_split, random_state=42)

4. Model Architecture

```
model = Sequential([
    Conv2D(64, (3,3), activation='relu', input_shape=(150,150,3)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),
    Dropout(0.25),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.25),
    Dense(2, activation='softmax')    # 2 classes: bike vs bicycle
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

Training uses early stopping on validation loss (patience=3, restores best weights) for up to 10 epochs.

5. Results

Metric	Value
Test Loss	0.150
Test Accuracy	95.39%

6. Prediction

A helper section loads a new image, resizes/normalizes it, and predicts its class along with a confidence score. A configurable THRESHOLD determines whether the model is confident enough to return a label, or reports "Image Not Recognized / Not Found" if it isn't.

```
prediction = model.predict(img)
predicted_class = np.argmax(prediction, axis=1)[0]
confidence = np.max(prediction) * 100
```

Usage

1. Clone the repo and open the notebook:

```
git clone <your-repo-url>
cd <your-repo>
jupyter notebook Bike_or_Bicycle_Detection_CNN.ipynb
```

2. Update `base_folder` in the data-loading cell to point to your image directory.

3. Run all cells to train the model.

4. Update `img_path` in the prediction section to test on your own image.

Possible Improvements

- Collect a larger, more diverse raw dataset instead of relying heavily on augmentation
- Use transfer learning (e.g., MobileNetV2, ResNet50) for stronger feature extraction
- Add a proper validation set separate from the test set
- Track experiments with TensorBoard
- Wrap prediction in a simple Flask/Streamlit app for demo purposes